

Novelty Detection for Multi-label Stream Classification

Joel D. Costa Júnior^{*§}, Elaine R. Faria[†], Jonathan A. Silva[‡], João Gama[§] and Ricardo Cerri^{*§}

^{*}Department of Computer Science (DC)

Federal University of São Carlos (UFSCar), São Carlos, Brazil

joel.costa@ufscar.br and cerri@ufscar.br

[†]Federal University of Uberlândia, Uberlândia, Brazil

elaine@ufu.br

[‡] Federal University of Mato Grosso do Sul, Ponta Porã, Brazil

jonathan.andrade@ufms.br

[§]Institute for Systems and Computer Engineering, Technology and Science (INESC TEC)

University of Porto (UP), Porto, Portugal

jgama@fep.up.pt

Abstract—In *Multi-Label Stream Classification* (MLSC) examples arriving in a stream can be simultaneously classified into multiple classes. This is a very challenging task, especially considering that new classes can emerge during the stream (*Concept Evolution*), and known classes can change over time (*Concept Drift*). In real situations, these characteristics come together with a scenario with *Infinitely Delayed Labels*, where we can never access the true class labels of the examples to update classifiers. In order to overcome these issues, this paper proposes a new method called *Multi-label learning Algorithm for Data Streams with Binary Relevance transformation* (MINAS-BR). Our proposal uses a new *Novelty Detection* (ND) procedure to detect concept evolution and concept drift, being updated in an unsupervised fashion. We also propose a new methodology to evaluate MLSC methods in scenarios with *Infinitely Delayed Labels*. Experiments over synthetic data sets attested the potential of MINAS-BR, which was able to adapt to different concept drift and concept evolution scenarios, obtaining superior or competitive performances in comparison to literature baselines.

Index Terms—Concept evolution, Infinitely delayed labels, Multi-label stream classification, Novelty detection

I. INTRODUCTION

In Machine Learning, conventional algorithms do not deal efficiently with continuously generated data in non-stationary scenarios, the so called *Data Streams* (DS). Thus, new methods must be created to deal with changes in data. They also must not store data, since the stream is possibly infinite [1]. In many DS classification problems, examples can be classified into multiple classes simultaneously. This is a very challenging task called *Multi-Label Stream Classification* (MLSC) [2], and has yet not been well-investigated in the literature.

In DS classification, *Novelty Detection* (ND) is the task of detecting new patterns in the arriving examples - which differ in some aspect from previously observed ones - and use them to update the decision models [1]. When applying ND in DS classification, we can detect new patterns, which may indicate

the emergence of new concepts (classes), changes in known concepts (concept drift), and also the presence of outliers.

In *Multi-Label Stream Classification* (MLSC) problems, the ND task is still an open problem. Very few authors have looked explicitly at important DS characteristics, such as the *Infinitely Delayed Labels*, where we can never access the true labels of the examples to update decision models [3], and the *Concept Evolution*, which is the emergence of new classes during the stream [4]. Motivated by these open issues, in this paper we propose a new method for MLSC able to detect concept evolution and concept drift, applying a ND procedure to update the model in an unsupervised fashion. Our new method is called *Multi-label learning Algorithm for Data Streams with Binary Relevance transformation* (MINAS-BR), and deals with MLSC applying the *Binary Relevance* (BR) problem transformation strategy [5], creating one decision model for each class. The method is an extension of the original MINAS [4] proposed for multi-class DS classification.

Besides proposing a new method for MLSC which is updated without the need of true labels of examples, neither external feedback, this paper presents other major contributions: i) a ND procedure able to detect and learn extensions of known classes (concept drift), and also the simultaneously emergence of more than one class (concept evolution); and ii) a novel methodology to evaluate MLSC methods, which considers unknown examples, and also which is able to assign new discovered patterns to classes.

The remainder of this paper is organized as follows. Section II surveys related work, while Section III describes our proposal. Section IV presents data sets, baseline methods, and parameter settings. Section V presents and discusses our experimental results, while conclusions and future works are presented in Section VI.

II. RELATED WORK

This section briefly presents some of the main MLSC methods in the literature. Some of them consider that the

This work was supported by the Brazilian research agencies FAPESP (grants #2017/11513-0, #2018/11321-6 and #2018/19829-9 - São Paulo Research Foundation (FAPESP)), CNPq, and CAPES - Finance Code 001.

true labels of the examples are available, and a few present mechanisms to update models with delayed labels.

Read et. al. [2] used problem transformation strategies with incremental base-classifiers. They used an ensemble that adapts to concept drift discarding the oldest or powerless model, rebuilding a new one. In the same work, the authors used *ADWIN Bagging* to detect concept drifts, and rebuild models only when necessary. The same authors proposed the *Multi-Label Hoeffding Tree* (MLHT), which uses multi-label Information Gain, running a *Pruned Sets* (PS) strategy with Naive-Bayes as base classifier in the leaves of the tree. In [6] the authors proposed an extended *Binary Relevance* (BR) using *Random Reference Classifiers* [7] with a *local fuzzy confusion matrix*, aiming at improving an ensemble of BR classifiers.

Osojnik et. al. [8] adapted a multi-target regression method. However, the method does not adapt to concept drifts and has high computational complexity. In [9] the authors adopted the same strategy. They proposed two methods, *Multi-Label AM-Rules* (ML-AMR) and *Multi-Label Random Rules* (ML-RR), both based on problem transformation strategies, but also having high computation complexity. To deal with computational complexities and memory consumption problems, Ahmadi and Kramer [10] recently proposed a label compression method for MLSC. They combine dependent labels into pseudo labels, and train one classifier for each of them.

Wang et. al. [11] deal with delayed labels using a label-based ensemble with Active Learning in order to continuously refine models. It is argued that individually updating classifiers can preserve information of classes which do not change when concept drift is detected, helping in updating models related to classes that are actually changing. However, the authors did not consider label dependencies.

Zhu et. al. [12] deal with concept evolution and infinitely delayed labels dividing the MLSC into three processes: *classification*, where a Pairwise Label Ranking classifier is proposed as a combination of binary linear classifiers, with a function to minimize the pairwise label ranking loss; *detection*, where an extension of the *Isolation Forest* method is proposed together with a clustering procedure to detect examples which may represent the emergence of new classes; and *updating*, where a new classifier is built for each new class according to an optimization function. In spite of concept evolution adaption, the method is unsuitable for concept drifts.

III. MINAS-BR

Our method considers that we have an initial set of labelled examples. Thus, it has two phases, offline and online.

A. Offline Phase

The MINAS-BR offline phase applies a *Binary Relevance* (BR) transformation to the problem. The method receives as input a training multi-label data set D_{tr} , formed by examples (X_i, Y_i) , with X_i the set of attributes of an example, and Y_i the corresponding set of classes. Given that D_{tr} has m training classes, the method then divides D_{tr} into m subsets Q_{y_j} , one for each training class y_j , $j = 1, \dots, m$. After creating the

initial subsets, the k-means algorithm is applied to each subset, thus generating models B_{y_j} , formed by a set of k micro-clusters MC_i , with $i = \{1, \dots, k\}$. The minimum number of examples in a micro-cluster is three. We chose this value based on the original MINAS [4] method. A micro-cluster MC has some components: its linear sum of the examples (LS), its quadratic sum of the examples (SS), its number of examples (n), the timestamp of the last example it classified (t), and its associated class label (y).

After applying k-means to each subset Q_{y_j} , each corresponding set of micro-clusters B_{y_j} is used as a decision model for the class y_j . As a result, the method returns a set M , containing m decision models, one for each training class y_j . The offline phase also returns the label cardinality z of the data set used in this phase. This is a parameter that will be used in the online phase.

B. Online Phase

The online phase is similar to the original MINAS method. However, here we build one model for each class, and arrival examples can be simultaneously classified by all models.

The online phase, described in Algorithm 1, verifies, for each new arrival example, which model B_{y_j} in M classifies it (step 8). For such, we compute the Euclidean distance ($dist$) between each new example and the centroids of each micro-cluster in B_{y_j} (step 9), getting, among all micro-clusters, the closest one ($MC_{closest}$) for each model. Given the closest micro-cluster of each model B_{y_j} , if its corresponding distance $dist$ is less or equal to the radius of the micro-cluster, we can classify the example with the label y_j corresponding to model B_{y_j} (step 11). If the example can not be classified by any decision model, it is labelled as *unknown* and is sent to a short-time memory (*ShortMem*).

Our *Novelty Detection* (ND) procedure (Algorithm 2) starts when the short-time memory reaches its storage limit (Θ , a user-defined parameter (Algorithm 1 step 17)). This procedure creates new micro-clusters using the unknown examples stored in the short-time memory, and then it verifies whether these micro-clusters represent *extensions* of known classes, leading to the update of the existing models, or represent *Novelty Patterns* (NPs), leading to the construction of new models.

The ND procedure creates new micro-clusters running the k-means algorithm together with the Leader algorithm on the short-time memory. We set the Leader algorithm parameter as the largest micro-clusters' radius r (step 3) of all models in M . These micro-clusters are assessed by the methodology proposed in Faria et. al. [4], which considers the well-known Silhouette Width Criterion value and the cardinality of the micro-clusters. A micro-cluster is valid if two conditions are satisfied: if it contains at least three examples, and its silhouette value is greater than zero (Algorithm 2, step 7).

After validating the new micro-clusters, we have to decide whether they represent extensions of the existing model or they represent NPs. For this, for each decision model B_{y_j} , we calculate the Euclidean distance $dist$ between the centroid of a new micro-clusters MC and the centroid of its closest

Algorithm 1: MINAS-BR online phase

Data: M : set of m decision models
 DS : multi-label data stream
 Θ : short-time memory maximum size
 ω : windows size
 $\mathcal{F}$: factor to calculate the NP threshold
 z : label cardinality of the training data set

```

1 begin
2   ShortMem  $\leftarrow \emptyset$ 
3    $t \leftarrow 0$  // timestamp of the arrival example
4   countNP  $\leftarrow 0$  // Number of existing models for NPs
5   foreach example  $X_i$  in  $DS$  do
6      $Y_i \leftarrow \emptyset$  // The labels of the new example
7      $t \leftarrow t + 1$ 
8     foreach model  $B_{y_j}$  in  $M$  do
9        $\{dist, MC_{closest}\} \leftarrow \text{getClosestMicroCluster}(X_i, B_{y_j})$ 
10      if  $dist \leq \text{getRadius}(MC_{closest})$  then
11        // Constructing the label set for  $X_i$ 
12         $Y_i \leftarrow Y_i \cup y_j$ 
13      end
14    end
15    if  $Y_i = \emptyset$  then
16      // No models in  $M$  classified  $X_i$ 
17       $Y_i \leftarrow \text{unknown}$ 
18      ShortMem  $\leftarrow \text{ShortMem} \cup (X_i, Y_i)$ 
19      if  $|ShortMem| \geq \Theta$  then
20        NDprocedure( $M, ShortMem, \mathcal{F}, z, t, countNP$ )
21      end
22    end
23    if  $(timeStamp \bmod \omega) = 0$  then
24      removeOldExamples( $ShortMem, \omega$ )
25      removeOldMicroClusters( $M, \omega$ )
26    end
27  end
28 end

```

micro-cluster $MC_{closest}$ in B_{y_j} . If $dist$ is less or equal than the standard deviation of the distances between the examples and the centroid of the $MC_{closest}$, multiplied by a factor $\mathcal{F}$, MC is considered an extension, otherwise, it is considered a NP.

Given the overlap between classes in multi-label learning, simply using only the distances between micro-clusters cannot be sufficient to determine whether they represent extensions or NPs. Thus, we use the label cardinality of the training data set as a parameter to decide whether a new micro-cluster differs too much from the ones that form the current models. Thus, a final decision is made by counting how many models considered a micro-cluster MC as an extension. If this counting is higher than the training set label cardinality z , then MC is added to all original models that considered it as an extension (Algorithm 2, step 18). These extensions represent concept drifts in the data. Thus, we use them to update the current models, adapting them to these concept drifts. If z is higher than the counting, the MC is considered to represent a NP, and a new model must be created to represent this NP.

When a MC is considered to represent a NP, we create a new model to represent this NP by putting together MC and copies of all the closest micro-clusters of the models Ext that considered MC as an extension (Algorithm 2, step 26). The micro-clusters of the new model will receive a new sequential label ($NP_1, NP_2, \dots$). Thus, if the current set of models M has $countNP$ decision models created for NPs, the next new decision model constructed will have label $NP_{countNP+1}$. Just created decision models represent the emergence of new

Algorithm 2: NDprocedure - updates decision models

Data: M : set of m decision models
 $ShortMem$: short-time memory
 $\mathcal{F}$: factor to calculate the NP threshold $\mathcal{T}$
 z : training data set's label cardinality
 t : timestamp // timestamp of the new micro-clusters
 $countNP$: number of existing models for NPs

```

1 begin
2    $r \leftarrow \text{getLargestRadius}(M)$ 
3    $B_{aux} \leftarrow \text{kMeansWithLeader}(ShortMem, r)$ 
4   foreach micro-cluster  $MC$  in  $B_{aux}$  do
5      $Ext \leftarrow \emptyset$  // Models to be extended
6     if  $\text{getNumberExamples}(MC) > 3$  AND  $\text{silhouette}(MC) > 0$  then
7       foreach model  $B_{y_j}$  in  $M$  do
8          $(dist, MC_{closest}) \leftarrow \text{closestMicroCluster}(MC, B_{y_j})$ 
9          $\mathcal{T} \leftarrow \text{getStdDev}(MC_{closest}) * \mathcal{F}$ 
10        if  $dist \leq \mathcal{T}$  then
11           $Ext \leftarrow Ext \cup B_{y_j}$ 
12        end
13      end
14    end
15    if  $|Ext| > z$  then
16      foreach  $B_{y_j}$  in  $Ext$  do
17         $y \leftarrow y_j$  // MC now has class  $y_j$ 
18         $B_{y_j} \leftarrow B_{y_j} \cup MC$ 
19      end
20    end
21    else
22      countNP  $\leftarrow countNP + 1$ 
23      // MC now has class  $NP_{countNP}$ 
24       $y \leftarrow NP_{countNP}$ 
25       $B_{NP} \leftarrow \emptyset$  // a new model with label  $y$ 
26       $B_{NP} \leftarrow B_{NP} \cup MC$  // put MC in  $B_{NP}$ 
27      foreach  $B_{y_j}$  in  $Ext$  do
28         $MC_{closest} \leftarrow \text{copyClosestMicroCluster}(MC, B_{y_j})$ 
29        //  $MC_{closest}$  now has class  $NP_{countNP}$ 
30         $y \leftarrow NP_{countNP}$ 
31         $B_{NP} \leftarrow B_{NP} \cup MC_{closest}$ 
32      end
33       $M \leftarrow M \cup B_{NP}$ 
34    end
35  end
36 end

```

classes in the stream. Recall that in both Algorithms 1 and 2, all MC s components $\{LS, SS, n, t, y\}$ are calculated with the creation of the MC s. For ease of visualization, we only show when the class y of a MC is updated. After creating models, these are added to the main set of models M . The updated set of models is now able to classify new incoming examples. Also, the models are able to verify whether new-formed micro-clusters are extensions, or they represent NPs.

As in the MINAS, to avoid the influence of obsolete information for updating models, from time to time our MINAS-BR online phase clears the short-time memory (Algorithm 1, step 22), and deletes micro-clusters which have not classified examples during a user-defined period of time (Algorithm 1, step 23). This user-defined parameter ω (Algorithm 1, step 21) defines how often to execute this cleaning procedure.

IV. EXPERIMENTAL SETUP

A. Data sets and baseline methods

We generated four synthetic data sets using a *Radial Basis Function* (RBF) from the *Massive Online Analysis framework* (MOA) [13]. In the data sets, each class is represented by possibly overlapped clusters of examples. For each data set,

Table I¹ shows the total number of examples ($|DS|$), the number of attributes ($|X|$), the number of known labels ($|L_{tr}|$), the number of novelty classes ($|L_{new}|$), the label cardinality (z), and the interval in number of examples ($speed$) between consecutive displacement of clusters in space.

TABLE I
CHARACTERISTICS OF THE SYNTHETIC DATA SETS.

Name	$ DS $	$ X $	$ L_{tr} $	$ L_{new} $	z	$speed$
MOA-3C-5C-2D	100,000	2	3	2	1.76	2,000
MOA-5C-7C-2D	100,000	2	5	2	1.54	1,500
MOA-5C-7C-3D	100,000	3	5	2	1.37	1,500
4CRE-V2	146,000	2	2	2	1.19	1,000

In the MINAS-BR offline phase, we used the first 10% of the stream to create the model. The remaining examples were used for testing in the online phase. Therefore, we consider that the set of known training classes of the problem is formed by the classes labelling the first 10% of the stream. All the different classes labelling examples belonging to the other 90% of the stream are considered novelty classes.

We compare our method against the literature methods presented in Table II. We divide the baseline methods between upper bounds (E_aBR , E_aCC and E_aMLHT) and lower bounds (CC , PS , $MLHT$ and $iSOUTree$). We use the subscript a to denote the methods which apply *ADaptive WINdow* (ADWIN) as a concept drift detector. All methods are available in the MOA framework.

The upper bound methods are ensembles of classifiers, and always have access to the true labels of the examples, updating their methods incrementally. Also, every time a concept drift is detected using the ADWIN algorithm, these methods rebuild their worst model in the ensemble. The lower bound methods, on the other hand, simulate a scenario where the true labels of the examples are never available. Thus, these methods never have an external feedback, and never update their models. They are only trained considering the first available 10% of the data and are never updated.

TABLE II
MLSC BASELINE METHODS AND RELEVANT REFERENCES.

Acronym	Algorithm	Reference
CC	Classifiers Chains	[14]
PS	Pruned Sets	[15]
$MLHT$	Multi-label Hoeffding Tree with PS at the leaves	[2]
$iSOUTree$	incremental Structured OUtput Prediction tree	[8]
E_aBR	Ensembles of Binary Relevance	[14]
E_aCC	Ensembles of Classifier Chains	[15]
E_aMLHT	Ensembles of <i>Multi-Label Hoeffding Tree</i> (MLHT)	[2]

B. Evaluation Methodology

The MINAS-BR deals with problems having infinitely delayed labels. Thus, when a new class emerges, it is impossible to update our model in a supervised way, since we cannot

¹The original version of the 4CRE-V2 data set is available in <https://sites.google.com/site/nonstationaryarchive/home>

define what are these new emergence classes, *i.e.*, the novel detected patterns come without any label. We are only able to see that they differ from the existing examples. To overcome the aforementioned problem, we presented in Algorithm 2 a ND unsupervised mechanism to create NPs that may represent the emergence of new classes. However, it is not possible to ascertain what are these classes, *i.e.*, what specific class a NP represents. We also can not detect the number of NPs representing each novel class. As a consequence, we can not apply traditional evaluation measures directly.

To perform a proper evaluation, we need first to associate the NPs to classes of the problem. Only then we can compute the evaluation measures. Based on the methodology proposed by Faria et. al. [16], we developed a new methodology that aims to associate a NP to a novelty class based on their degree of dependency. To measure the dependency degree between a NP_i and a novelty class y_{new_j} , we build a contingency matrix as shown in Table III. Recall that y_{new_j} is a new class that may appear during the stream, and will appear until the end of the stream. For evaluation purposes, we have now to decide which NP is going to be associated to this class. This association process is performed whenever necessary to evaluate the method (user defined).

TABLE III
CONTINGENCY MATRIX FOR NP_i AND CLASS y_{new_j} .

	NP_i	$\neg NP_i$
y_{new_j}	a	b
$\neg y_{new_j}$	c	d

The contingency matrix shown in Table III gives, for every NP_i and novelty class y_{new_j} : i) the number of times a NP_i appears together with a class y_{new_j} (a); ii) the number of times a class y_{new_j} appears, but a NP_i does not appear (b); iii) the number of times a NP_i appears but a class y_{new_j} does not appear (c); and iv) the number of times a NP_i does not appear and a class y_{new_j} does not appear either (d). Given this contingency matrix, we can compute the score of the matrix using the F1 measure (Equation 1). We chose the F1 measure because it ignores the element d of the contingency table, since we are not interested in these cases, and they can produce misleading in results.

$$F1 = \frac{a}{a + \frac{b+c}{2}} \quad (1)$$

The construction of a contingency matrix and computation of the corresponding F1 measure is performed for every NP_i that has no association with any class. A NP_i will be associated to the novelty class y_{new_j} which results in the higher dependency degree. If given a NP_i , the F1 obtained for all novelty classes is equal to 0, then we consider the method does not have enough information to associate the NP_i to any class. In this case, it remains without association until the next evaluation process.

When a NP_i is associated with a class y_{new_j} , all examples predicted as NP_i are now said to be classified as y_{new_j} . As a result, we can apply conventional evaluation measures. We must also consider the presence of *unknown* examples during the evaluation process. These are examples that were not classified by any model, or were not used for the construction of any model, possibly because their corresponding micro-clusters were not valid ones. In order to measure how the number of *unknown* examples varies over time, we compute the *Macro Unknown Rate* (UnkRM) measure (Equation 2) in every evaluation window. Thus, we evaluate the performance of MINAS-BR using the traditional *Macro F-Measure* (F1M) (Equation 3), and UnkRM measure.

$$UnkRM = \frac{1}{|L|} \sum_{i=1}^{|L|} \frac{UNK_i}{N_i} \quad (2)$$

$$F1M = \frac{1}{|L|} \sum_{i=1}^{|L|} \frac{TP_i}{TP_i + \frac{FN_i + FP_i}{2}} \quad (3)$$

In Equation 2, UNK_j represents the number of *unknown* examples from the class y_j , and N_j represents the total number of examples from the class y_j . We chose macro-averaging measures because they better reflect the emergence of a new class, facilitating the assessment of the influence of the ND procedure over the stream.

C. Parameter Settings

Table IV presents the parameters and corresponding values used in the experiments with MINAS-BR. We defined these values based on the original MINAS [4]. All experiments were performed considering the variations presented in Table. IV

TABLE IV
PARAMETER SETTINGS FOR MINAS-BR

Parameters	Values used
k_{ini} : value for k-means in the initial offline training phase	1%, 5%, 10% and 30% of the number of examples to be grouped
Minimum number of examples for a valid micro-cluster	3
Number of models which classify an example in the online phase	The number of models which can explain the example
Window size (ω) to forget outdated examples and to remove outdated micro-clusters	100, 500, 1000, 2000
Short-time memory maximum size (Θ)	30, 100, 500, 1000, 2000
Factor $\mathcal{F}$ to calculate the NP threshold $\mathcal{T}$	1.0, 1.1, 1.3, 1.5
Parameter to decide if micro-clusters are extensions or NPs	$ Ext > z$

V. RESULTS AND DISCUSSION

Our main goal in the experiments is to obtain better results than the lower bound methods. The upper bound methods are used to verify how is the MINAS-BR performance compared to methods that have access to the true labels of the examples to update their classifiers.

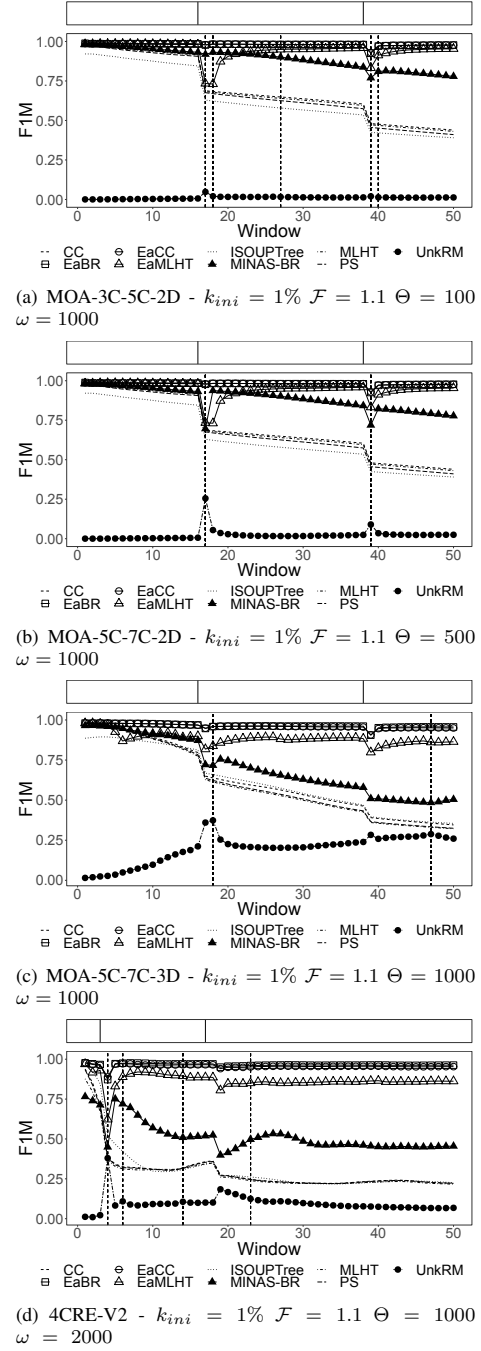


Fig. 1. Results over all data sets

Figure 1 presents the results over all data sets. We report F1M values across the entire stream using a window of size $w = N_{test}/50$ (the entire stream, except the first 10% used for training, is divided into 50 windows). The y-axis represents the F1M values, while the x-axis represents all 50 windows. The continuous vertical lines in the rectangles above the graphs

represent the windows where a new class emerges, and the dashed vertical lines crossing the graphs represent the exact windows where MINAS-BR builds models to represent NPs. Also, to evaluate the ND procedure, it is important to report the *Macro Unknown Rate* (UnkRM) together with the F1M values. Thus, with the F1M values for all methods, we report the MINAS-BR UnkRM values in the curves represented by gray dashes. For each data set, we show the best results obtained for all combinations of the parameter values in Table IV. These parameter values are shown in the caption of each graph.

In *MOA-3C-5C-2D*, *MOA-5C-7C-2D* and *MOA-5C-7C-3D*, the novelty classes emerge at windows 16 and 38, and in *4CRE-V2*, the novelty classes emerge at windows 3 and 17. The peaks for UnkRM indicate that examples from novelty classes are appearing. In this case, MINAS-BR uses these examples to construct models to represent NPs (dashed lines). In consequence, UnkRM must decrease, indicating that the method adapted well to the emergence of a new class. The performance was as expected for *MOA-3C-5C-2D* and *MOA-5C-7C-2D* (Figures 1(a) and 1(b), respectively). Especially in *MOA-3C-5C-2D*, MINAS-BR adapted very fast to the concept evolution. However, for *MOA-5C-7C-3D* and *4CRE-V2* (Figure 1(c) and 1(d)), there was a delay when detecting NPs. In the *MOA-5C-7C-3D* data set, when the first class emerged (window 16), MINAS-BR constructed NP models with a low number of examples. As a result of that, UnkRM slightly decreased indicating that a substantial number of examples was left into short-time-memory. The second emergent class (window 38) is too overlapped with other classes. This results in a continuously increase in UnkRM until models for NPs are constructed. We can see that MINAS-BR performed better than the lower bound methods in all data sets.

The *4CRE-V2* data set changes faster than the other data sets, i.e., it takes a fewer number of examples for a displacement of the clusters in space. Also, the clusters representing its four classes are simultaneously rotated and moved close and away from each other in space. This is a harder scenario to detect concept drifts and concept evolution. MINAS-BR detect well the first concept evolution (window 3), but created unnecessary NPs at windows 6 and 14, causing classification mistakes. This kept UnkRM stable while decreased F1M. As in *MOA-5C-7C-3D*, the second emergent class (window 17) also appears too overlapped to other classes. Therefore, MINAS-BR detected this novelty class only when its corresponding cluster moved away from the others.

The experiments performed showed that MINAS-BR could detect novelty classes while keeping (or slightly decreasing) its predictive power. Still, it outperformed the lower bound methods for almost all data sets investigated, and showed competitive results compared to upper bound methods in some situations.

VI. CONCLUSIONS AND FUTURE WORKS

This paper proposed MINAS-BR, a method to deal with Infinitely Delayed Labels in MLSC. It handles concept evolution and concept drifts applying a new ND procedure. We

also presented a new evaluation methodology that associates NPs to novelty classes using their degree of dependency.

The experimental results showed that MINAS-BR detected all novelty classes keeping (or slightly decreasing) its predictive power. Our method outperformed the lower bound methods for all data sets, and was competitive to the upper bounds in some situations. Also, differently from the upper bounds, our proposal has the advantage of being updated without the true labels of the examples and without any external feedback.

This paper opens several perspectives for future works, especially considering ND applied to MLSC, and evaluation methodologies. As future steps, we intend to use real multi-label data sets, investigate new strategies to automatically select the best threshold value to differentiate extensions from NPs, and improve the detection of overlapping novelty classes.

REFERENCES

- [1] J. Gama, *Knowledge discovery from data streams*. Chapman & Hall/CRC Data Mining and Knowledge Discovery Series, 2010.
- [2] J. Read, A. Bifet, G. Holmes, and B. Pfahringer, "Scalable and efficient multi-label classification for evolving data streams," *Machine Learning*, vol. 88, no. 1-2, pp. 243–272, 2012.
- [3] V. M. Souza, D. F. Silva, G. E. Batista, and J. Gama, "Classification of evolving data streams with infinitely delayed labels," in *Machine Learning and Applications (ICMLA), 2015 IEEE 14th International Conference on*. IEEE, 2015, pp. 214–219.
- [4] E. R. de Faria, A. C. P. de Leon Ferreira, J. Gama *et al.*, "Minas: multiclass learning algorithm for novelty detection in data streams," *Data Mining and Knowledge Discovery*, vol. 30, no. 3, pp. 640–680, 2016.
- [5] G. Tsoumakas, I. Katakis, and I. Vlahavas, "Mining multi-label data," in *Data mining and knowledge discovery handbook*. Springer, 2009, pp. 667–685.
- [6] P. Trajdos and M. Kurzynski, "Multi-label stream classification using extended binary relevance model," in *Trustcom/BigDataSE/ISPA, 2015 IEEE*, vol. 2. IEEE, 2015, pp. 205–210.
- [7] T. Woloszynski and M. Kurzynski, "A probabilistic model of classifier competence for dynamic ensemble selection," *Pattern Recognition*, vol. 44, no. 10-11, pp. 2656–2668, 2011.
- [8] A. Osojnik, P. Panov, and S. Džeroski, "Multi-label classification via multi-target regression on data streams," *Machine Learning*, vol. 106, no. 6, pp. 745–770, 2017.
- [9] R. Sousa and J. Gama, "Multi-label classification from high-speed data streams with adaptive model rules and random rules," *Progress in Artificial Intelligence*, pp. 1–11, 2018.
- [10] Z. Ahmadi and S. Kramer, "A label compression method for online multi-label classification," *Pattern Recognition Letters*, vol. 111, pp. 64–71, 2018.
- [11] P. Wang, P. Zhang, and L. Guo, "Mining multi-label data streams using ensemble-based active learning," in *Proceedings of the 2012 SIAM international conference on data mining*. SIAM, 2012, pp. 1131–1140.
- [12] Y. Zhu, K. M. Ting, and Z. Zhou, "Multi-label learning with emerging new labels," *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 10, pp. 1901–1914, 2018.
- [13] A. Bifet, G. Holmes, R. Kirkby, and B. Pfahringer, "Moa: Massive online analysis," *Journal of Machine Learning Research*, vol. 11, no. May, pp. 1601–1604, 2010.
- [14] J. Read, B. Pfahringer, G. Holmes, and E. Frank, "Classifier chains for multi-label classification," *Machine learning*, vol. 85, no. 3, pp. 333–359, 2011.
- [15] J. Read, B. Pfahringer, and G. Holmes, "Multi-label classification using ensembles of pruned sets," in *Data Mining, 2008. ICDM'08. Eighth IEEE International Conference on*. IEEE, 2008, pp. 995–1000.
- [16] E. R. de Faria, I. R. Goncalves, J. Gama, A. C. P. de Leon Ferreira *et al.*, "Evaluation of multiclass novelty detection algorithms for data streams," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 11, pp. 2961–2973, 2015.